

Hindernisse für ein Lama

Ein Lama möchte durch die Anden-Hochebene reisen. Es hat eine Karte der Hochebene in Form eines Gitters aus $N \times M$ quadratischen Zellen. Die Zeilen der Karte sind von 0 bis $N - 1$ von oben nach unten nummeriert, und die Spalten sind von 0 bis $M - 1$ von links nach rechts nummeriert. Die Zelle der Karte in der Zeile i und der Spalte j ($0 \leq i < N, 0 \leq j < M$) wird mit (i, j) bezeichnet.

Das Lama hat das Klima der Hochebene untersucht und festgestellt, dass alle Zellen in einer Zeile der Karte jeweils die gleiche **Temperatur** und alle Zellen in einer Spalte der Karte jeweils die gleiche **Luftfeuchtigkeit** haben. Das Lama stellt dir zwei Listen von ganzen Zahlen T und H mit der Länge N bzw. M zur Verfügung. Dabei gibt $T[i]$ ($0 \leq i < N$) die Temperatur der Zellen in der Zeile i an, und $H[j]$ ($0 \leq j < M$) die Luftfeuchtigkeit der Zellen in der Spalte j an.

Das Lama hat auch die Flora der Hochebene studiert und festgestellt, dass eine Zelle (i, j) genau dann **frei von Vegetation** ist, wenn ihre Temperatur größer ist als ihre Luftfeuchtigkeit, formal $T[i] > H[j]$.

Das Lama kann die Hochebene nur auf **gültigen Pfaden** durchqueren. Ein gültiger Pfad ist eine Abfolge verschiedener Zellen, die die folgenden Bedingungen erfüllen:

- Jedes Paar aufeinanderfolgender Zellen im Pfad hat eine gemeinsame Seite.
- Alle Zellen des Pfades sind frei von Vegetation.

Deine Aufgabe ist es, Q Anfragen zu beantworten. Für jede Anfrage sind vier ganze Zahlen: L, R, S , und D gegeben. Du sollst herausfinden, ob es einen gültigen Pfad gibt, der den folgenden Regeln entspricht:

- Der Pfad beginnt in der Zelle $(0, S)$ und endet in der Zelle $(0, D)$. Es ist garantiert, dass sowohl $(0, S)$ als auch $(0, D)$ frei von Vegetation sind.
- Alle Zellen des Pfades liegen zwischen oder auf den Spalten L und R .

Implementierungsdetails

Implementiere zwei Funktionen:

```
void initialize(std::vector<int> T, std::vector<int> H)
```

- T : eine Liste der Länge N , die die Temperatur für jede Zeile angibt.
- H : eine Liste der Länge M , die die Luftfeuchtigkeit für jede Spalte angibt.
- Diese Funktion wird für jeden Testfall genau einmal aufgerufen, und zwar vor dem ersten Aufruf der Funktion `can_reach`.

```
bool can_reach(int L, int R, int S, int D)
```

- L, R, S, D : ganze Zahlen, die eine Anfrage beschreiben.
- Diese Funktion wird für jeden Testfall Q mal aufgerufen.

Diese Funktion soll genau dann `true` zurückgeben, wenn es einen gültigen Pfad von der Zelle $(0, S)$ zur Zelle $(0, D)$ gibt, so dass alle Zellen im Pfad zwischen oder auf den Spalten L und R liegen.

Einschränkungen

- $1 \leq N, M, Q \leq 200\,000$
- $0 \leq T[i] \leq 10^9$ für jedes i ($0 \leq i < N$).
- $0 \leq H[j] \leq 10^9$ für jedes j ($0 \leq j < M$).
- $0 \leq L \leq R < M$
- $L \leq S \leq R$
- $L \leq D \leq R$
- Die beiden Zellen $(0, S)$ und $(0, D)$ sind frei von Vegetation.

Subtasks

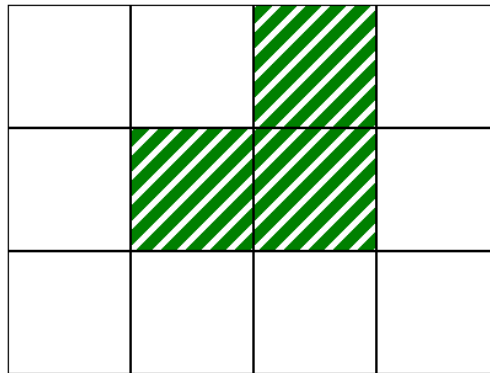
Subtask	Punkte	Zusätzliche Einschränkungen
1	10	$L = 0, R = M - 1$ für jede Anfrage. $N = 1$.
2	14	$L = 0, R = M - 1$ für jede Anfrage. $T[i - 1] \leq T[i]$ für alle i ($1 \leq i < N$).
3	13	$L = 0, R = M - 1$ für jede Anfrage. $N = 3$ und $T = [2, 1, 3]$.
4	21	$L = 0, R = M - 1$ für jede Anfrage. $Q \leq 10$.
5	25	$L = 0, R = M - 1$ für jede Anfrage.
6	17	Keine zusätzlichen Einschränkungen.

Beispiel

Betrachte den folgenden Aufruf:

```
initialize([2, 1, 3], [0, 1, 2, 0])
```

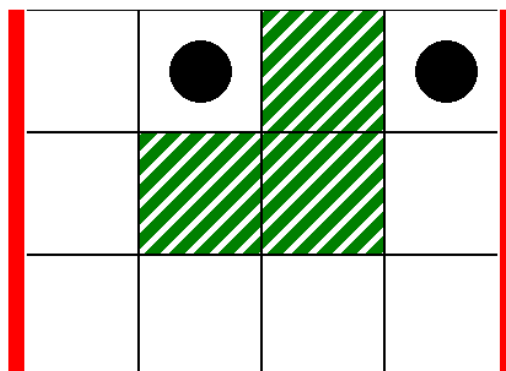
Dies entspricht der folgenden Karte. Die weißen Zellen sind frei von Vegetation.



Betrachte als erste Frage den folgenden Aufruf:

```
can_reach(0, 3, 1, 3)
```

Dies entspricht der Situation in der folgenden Abbildung, in der die dicken vertikalen Linien den Bereich der Spalten von $L = 0$ bis $R = 3$ anzeigen und die schwarzen Kreise die Start- und Endzellen angeben:



In diesem Fall kann das Lama von der Zelle $(0, 1)$ zur Zelle $(0, 3)$ über den folgenden gültigen Pfad gelangen:

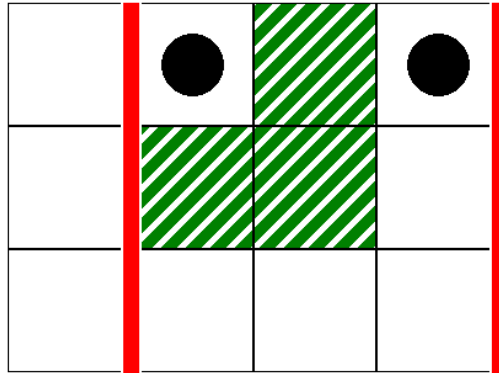
$(0, 1), (0, 0), (1, 0), (2, 0), (2, 1), (2, 2), (2, 3), (1, 3), (0, 3)$

Daher sollte dieser Aufruf `true` zurückgeben.

Betrachten wir als zweite Frage den folgenden Aufruf:

```
can_reach(1, 3, 1, 3)
```

Dies entspricht dem Szenario in der folgenden Abbildung:



In diesem Fall gibt es keinen gültigen Pfad von der Zelle $(0, 1)$ zur Zelle $(0, 3)$, so dass alle Zellen im Pfad innerhalb der Spalten 1 bis einschließlich 3 liegen. Daher sollte dieser Aufruf `false` zurückgeben.

Beispiel-Grader

Eingabeformat:

```
N M
T[0] T[1] ... T[N-1]
H[0] H[1] ... H[M-1]
Q
L[0] R[0] S[0] D[0]
L[1] R[1] S[1] D[1]
...
L[Q-1] R[Q-1] S[Q-1] D[Q-1]
```

Hier geben $L[k]$, $R[k]$, $S[k]$ und $D[k]$ ($0 \leq k < Q$) die Parameter für jeden Aufruf von `can_reach` an.

Ausgabeformat:

```
A[0]
A[1]
...
A[Q-1]
```

Hier ist $A[k]$ ($0 \leq k < Q$) 1, wenn der Aufruf `can_reach(L[k], R[k], S[k], D[k])` `true` zurückgegeben hat und 0 andernfalls.